

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Báo Cáo Đồ Án 2: CI/CD, GitOps, Service Mesh và Observability

Đề tài: Triển khai CI/CD, GitOps, Service Mesh và Observability cho
hệ thống YAS — Yet Another Shop

Môn học: DevOps

Sinh viên thực hiện:

Ngô Gia An (23120205)

Nguyễn Bảo An (23120207)

Dương Tuấn Anh (23120208)

Trương Nhật Đạt (23120231)

Ngày 9 tháng 7 năm 2026

Mục lục

1	Thông Tin Nhóm	1
2	Yêu Cầu Đồ Án	1
3	Phân Công Công Việc	2
4	Tổng Quan Kiến Trúc Pipeline	2
5	Hạ Tầng Triển Khai	3
6	CI/CD Với Jenkins	3
7	GitOps Và Kustomize	4
8	Service Mesh Với Istio, Security Policy Và Kiali	4
8.1	Mục tiêu và phạm vi	4
8.2	Cấu hình Istio	5
8.3	Sidecar injection và trạng thái Istio system	5
8.4	mTLS và DestinationRule	7
8.5	Retry policy	8
8.6	AuthorizationPolicy và kiểm thử allow/deny	9
8.7	Kiali visualization	10
8.8	Kiểm chứng Istio ingress	12
8.9	Kết luận phần Service Mesh	12
9	Observability	12
10	Kết Luận	13

Danh sách hình vẽ

1	Luồng triển khai tổng quát của Project 2	3
2	Kiali Istio Config hiển thị các resource đại diện trong namespace dev	5
3	Các thành phần Istio, Kiali, Prometheus và Grafana đang chạy trong istio-system	6
4	Các namespace triển khai có label istio-injection=enabled	6
5	Pod trong dev có container ứng dụng và sidecar istio-proxy	6
6	Các pod ứng dụng trong dev ở trạng thái 2/2 Running	7
7	PeerAuthentication STRICT và DestinationRule ISTIO_MUTUAL trong namespace dev	8
8	VirtualService tax-retry cấu hình retry cho lỗi tạm thời	9
9	Kiểm thử AuthorizationPolicy: request hợp lệ trả 200, request không hợp lệ bị chặn với 403 RBAC	10
10	Kiali Overview hiển thị các namespace trong mesh	11
11	Kiali Service Graph thể hiện topology, traffic và mTLS trong namespace dev	11
12	Kiểm chứng ingress: các endpoint chính đều trả HTTP 200	12

1 Thông Tin Nhóm

Bảng 1: Thông tin thành viên nhóm

Thành viên	MSSV	Phần phụ trách
Ngô Gia An	23120205	Hạ tầng GCP/K3s, ArgoCD, Jenkins agent và Observability
Nguyễn Bảo An	23120207	Jenkins CI/CD, build/push image và developer-build
Trương Nhật Đạt	23120231	GitOps manifest, Kustomize overlays và ArgoCD applications
Dương Tuấn Anh	23120208	Istio Service Mesh, security policy, Kiali và minh chứng vận hành

Link GitHub Repository: <https://github.com/com-suon-bi-cha/yas>

Repo GitOps manifest: <https://github.com/com-suon-bi-cha/gitops-manifest-k8s>

2 Yêu Cầu Đồ Án

Phần này tổng hợp các yêu cầu chính của đồ án và trạng thái hoàn thành tương ứng. Các nội dung được đối chiếu theo kết quả triển khai thực tế trên Jenkins, repo GitOps, ArgoCD và Kubernetes cluster.

Bảng 2: Đối chiếu yêu cầu chính của đồ án

Yêu cầu	Kết quả thực hiện	Trạng thái
CI cho các service trong scope	Jenkins build/test/push Docker image cho các workload ứng dụng được chọn	Đạt
CD lên Kubernetes	ArgoCD đồng bộ manifest từ repo GitOps vào namespace dev và staging	Đạt
GitOps manifest	Kustomize quản lý base và overlays cho dev , staging , developer-build	Đạt
Service Mesh nâng cao	Istio cấu hình mTLS, retry, AuthorizationPolicy, ingress routing và Kiali visualization cho môi trường dev ; staging có overlay Istio riêng	Đạt
Observability	Prometheus, Grafana, Loki, Tempo, Promtail và OpenTelemetry Collector được triển khai trong namespace observability	Đạt
Minh chứng vận hành	Screenshot Jenkins, ArgoCD, Kubernetes, Kiali và Observability được tổ chức trong thư mục báo cáo	Đạt

Phạm vi triển khai hiện tại gồm 16 workload ứng dụng:

- Core services: `product`, `cart`, `order`, `customer`, `inventory`, `tax`, `payment`.
- Supporting services: `media`, `search`, `location`, `sampledata`.
- Storefront: `storefront-bff`, `storefront-ui`.
- Backoffice và công cụ: `backoffice-bff`, `backoffice-ui`, `swagger-ui`.

Workload `kafka-connect` được triển khai như thành phần hỗ trợ CDC, không tính là service ứng dụng chính.

3 Phân Công Công Việc

Việc phân công được tổ chức theo các lớp kỹ thuật của hệ thống. Mỗi phạm vi có đầu ra độc lập, đồng thời phụ thuộc vào các phần còn lại để hình thành chuỗi CI/CD hoàn chỉnh.

Bảng 3: Phân công theo phạm vi kỹ thuật

Phụ trách	Phạm vi kỹ thuật	Sản phẩm chính
Ngô Gia An	Hạ tầng cloud, K3s, namespace, secret, ArgoCD, Jenkins agent và Observability	Cluster sẵn sàng cho CI/CD, GitOps, Service Mesh và hệ thống quan sát
Nguyễn Bảo An	Jenkins CI/CD, Docker image, developer-build và cập nhật image tag	Pipeline build/test/push và cơ chế triển khai theo branch
Trương Nhật Đạt	GitOps manifest, Kustomize, overlay môi trường và ArgoCD application	Repo manifest có thể đồng bộ vào dev, staging, developer-build
Dương Tuấn Anh	Istio Service Mesh, mTLS, AuthorizationPolicy, retry, Kiali và ingress verification	Cấu hình mesh cho dev, kiểm chứng allow/deny, topology và ingress

4 Tổng Quan Kiến Trúc Pipeline

Hệ thống YAS được triển khai theo mô hình CI/CD kết hợp GitOps. Source code ứng dụng và manifest triển khai được tách thành hai nguồn quản lý độc lập. Khi có thay đổi mã nguồn, Jenkins Multibranch Pipeline thực hiện build/test, đóng gói Docker image và push image lên Docker Hub.

Sau đó pipeline cập nhật image tag trong repo `gitops-manifest-k8s`. ArgoCD theo dõi repo GitOps và đồng bộ trạng thái mong muốn vào Kubernetes cluster.

GitHub → Jenkins CI/CD → Docker Hub → GitOps Manifest → ArgoCD → K3s Cluster

Hình 1: Luồng triển khai tổng quát của Project 2

Trong cluster, namespace `dev` và `staging` đều được quản lý bằng GitOps. Namespace `dev` là môi trường kiểm thử chính cho Service Mesh, còn `staging` có overlay Istio riêng để đáp ứng yêu cầu triển khai chính sách mesh ở môi trường tiền triển khai. Hai môi trường này có sidecar injection, mTLS, retry policy và AuthorizationPolicy. Namespace `developer-build` phục vụ triển khai tạm thời cho branch khi chạy job kiểm thử. Namespace `observability` chứa các thành phần quan sát hệ thống như Prometheus, Grafana, Loki, Tempo, Promtail và OpenTelemetry Collector.

5 Hạ Tầng Triển Khai

Phần hạ tầng cung cấp nền tảng thực thi cho toàn bộ pipeline. Cluster K3s chạy trên máy ảo GCP, các namespace chính gồm `dev`, `staging`, `developer-build`, `argocd`, `istio-system` và `observability`. ArgoCD được cài đặt để quản lý trạng thái triển khai theo GitOps. Các secret cần thiết như Docker registry credentials và kubeconfig được cấu hình để Jenkins có thể build, push image và cập nhật manifest.

Nội dung minh chứng cho phần này gồm trạng thái VM, node Kubernetes, namespace, ArgoCD applications và các service hạ tầng.

6 CI/CD Với Jenkins

Jenkins đảm nhiệm quy trình CI/CD cho các workload trong scope hiện tại. Pipeline thực hiện các bước chính: chọn service cần build, chạy kiểm tra cần thiết, build Docker image, push image lên Docker Hub với repository `bingsu1103/*`, sau đó cập nhật image tag trong repo GitOps. Cách tổ chức này giúp tách riêng mã nguồn ứng dụng và trạng thái triển khai, đồng thời giảm thao tác thủ công khi phát hành service mới.

Ngoài pipeline triển khai chính, nhóm còn cấu hình luồng `developer-build`. Luồng này cho phép

build và triển khai tạm thời service từ branch phát triển vào namespace **developer-build**. Deployment trong namespace này được scale về 0/0 khi không sử dụng để tiết kiệm tài nguyên cluster. Minh chứng cho phần này gồm Jenkins dashboard, pipeline stages, Docker Hub tags và kết quả **developer-build**.

7 GitOps Và Kustomize

Repo **gitops-manifest-k8s** là nguồn cấu hình triển khai cho Kubernetes. Repo này tổ chức manifest theo mô hình **base** và **overlays**. **base** định nghĩa Deployment, Service, ServiceAccount và cấu hình chung của từng workload. Các overlay **dev**, **staging** và **developer-build** điều chỉnh image tag, namespace, replica và cấu hình môi trường.

ArgoCD theo dõi repo GitOps và đồng bộ trạng thái mong muốn vào cluster. Khi Jenkins cập nhật image tag trong repo manifest, ArgoCD phát hiện thay đổi và rollout workload tương ứng. Cơ chế này bảo đảm quy trình CD có lịch sử rõ ràng, có thể truy vết qua commit GitOps và trạng thái ArgoCD.

Minh chứng cho phần này gồm cấu trúc repo GitOps, kết quả Kustomize render, trạng thái ArgoCD sync và workload trong từng namespace.

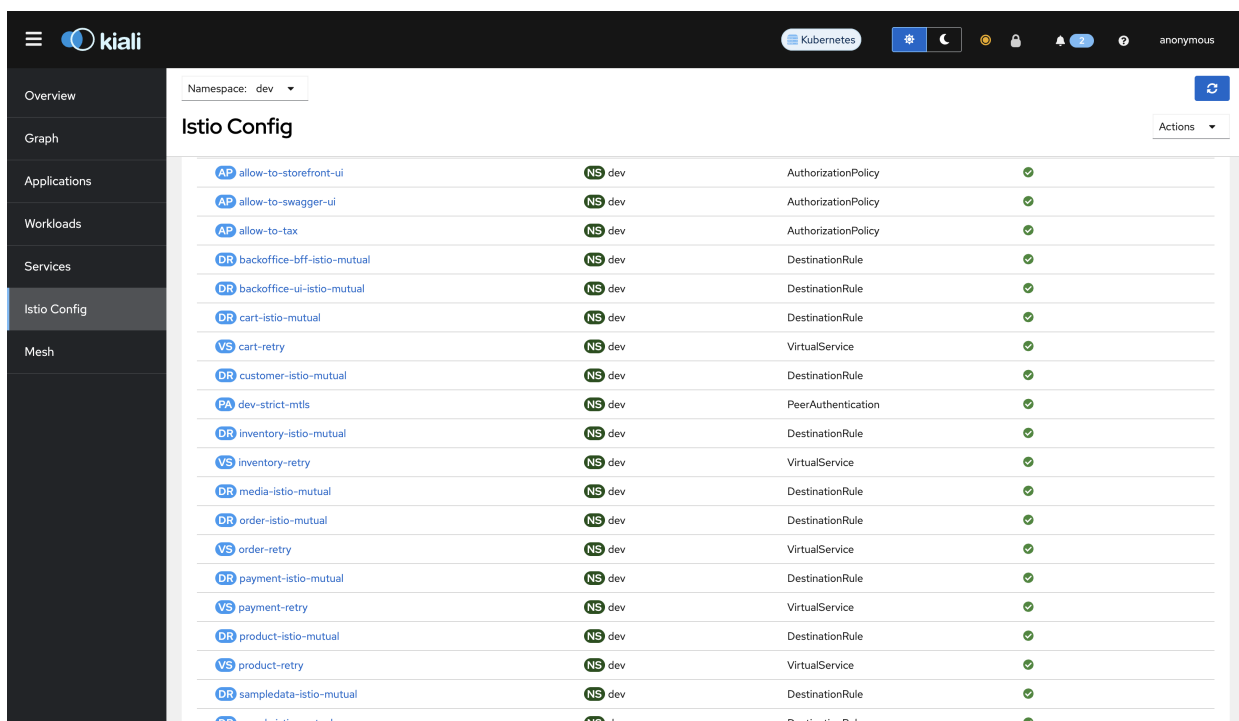
8 Service Mesh Với Istio, Security Policy Và Kiali

8.1 Mục tiêu và phạm vi

Phần Service Mesh tập trung vào cấu hình và kiểm chứng luồng giao tiếp giữa các service của hệ thống YAS bằng Istio. Phạm vi chính gồm ingress routing, sidecar injection, mTLS, DestinationRule, retry policy, AuthorizationPolicy, kiểm thử allow/deny và trực quan hóa bằng Kiali. Môi trường **dev** được dùng để kiểm chứng chi tiết các chính sách mesh. Môi trường **staging** được bổ sung overlay Istio riêng với mTLS, DestinationRule, retry policy và AuthorizationPolicy tương ứng để đáp ứng yêu cầu triển khai ở môi trường tiền sản xuất.

8.2 Cấu hình Istio

Cấu hình Istio được quản lý theo hướng GitOps trong repo manifest. Gateway và VirtualService ở lớp base định tuyến request từ ingress gateway tới Storefront, Swagger UI và các BFF/API tương ứng. Overlay `dev` và `staging` bổ sung các resource bảo mật và điều phối traffic, gồm `PeerAuthentication`, `DestinationRule`, `VirtualService` retry và `AuthorizationPolicy`. Các file staging được tách riêng tại `environments/staging/istio` để host và ServiceAccount principal trở đúng namespace `staging`.



The screenshot shows the Kiali Istio Config interface. The left sidebar contains navigation links: Overview, Graph, Applications, Workloads, Services, Istio Config (selected), and Mesh. The main panel displays a table of Istio resources in the `dev` namespace. The table has columns for resource type, name, namespace, and status. All resources are in the `dev` namespace and have a status of `OK`.

Resource Type	Name	Namespace	Status
AP	allow-to-storefront-ui	NS dev	OK
AP	allow-to-swagger-ui	NS dev	OK
AP	allow-to-tax	NS dev	OK
DR	backoffice-bff-istio-mutual	NS dev	OK
DR	backoffice-ui-istio-mutual	NS dev	OK
DR	cart-istio-mutual	NS dev	OK
VS	cart-retry	NS dev	OK
DR	customer-istio-mutual	NS dev	OK
PA	dev-strict-mtls	NS dev	OK
DR	inventory-istio-mutual	NS dev	OK
VS	inventory-retry	NS dev	OK
DR	media-istio-mutual	NS dev	OK
DR	order-istio-mutual	NS dev	OK
VS	order-retry	NS dev	OK
DR	payment-istio-mutual	NS dev	OK
VS	payment-retry	NS dev	OK
DR	product-istio-mutual	NS dev	OK
VS	product-retry	NS dev	OK
DR	sampledata-istio-mutual	NS dev	OK
DR	search-istio-mutual	NS dev	OK

Hình 2: Kiali Istio Config hiển thị các resource đại diện trong namespace `dev`

Hình trên cho thấy các resource Istio chính tồn tại trong namespace `dev` và có trạng thái hợp lệ. Các nhóm cấu hình đại diện gồm access policy, mTLS, routing và retry. Đây là bằng chứng trực quan cho việc cấu hình Istio đã được đồng bộ vào cluster.

8.3 Sidecar injection và trạng thái Istio system

Istio system, Kiali, Prometheus và Grafana được triển khai trong namespace `istio-system`. Namespace `dev`, `staging` và `developer-build` được gắn label `istio-injection=enabled` để Istio tự động inject sidecar vào pod ứng dụng.


```
bingsul103@instance-20260620-064227:~$ kubectl get pods -n istio-system
kubectl get svc -n istio-system
NAME                                READY    STATUS    RESTARTS   AGE
grafana-79fdf8f554-pvp4t            1/1      Running   0           21h
istio-egressgateway-f7bc49487-cdhmg 1/1      Running   0           21h
istio-ingressgateway-55dd967889-qj96t 1/1      Running   0           21h
istiod-7b6c674747-4z225             1/1      Running   0           21h
kiali-75ccc89fd-67bvb               1/1      Running   0           21h
prometheus-55567457cd-7brpd         2/2      Running   0           21h
NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
grafana                             ClusterIP      10.43.145.204  <none>          3000/TCP                               8d
istio-egressgateway                 ClusterIP      10.43.226.223  <none>          80/TCP,443/TCP                        14d
istio-ingressgateway                LoadBalancer  10.43.149.134  <pending>      15021:32047/TCP,80:31255/TCP,443:32542/TCP,31400:30965/TCP,15443:31602/TCP 14d
istiod                               ClusterIP      10.43.171.227  <none>          15010/TCP,15012/TCP,443/TCP,15014/TCP 14d
kiali                               ClusterIP      10.43.213.144  <none>          20001/TCP,9090/TCP                     8d
prometheus                           ClusterIP      10.43.78.38    <none>          9090/TCP                               8d
bingsul103@instance-20260620-064227:~$
```

Hình 3: Các thành phần Istio, Kiali, Prometheus và Grafana đang chạy trong istio-system

```
bingsul103@instance-20260620-064227:~$ kubectl get ns dev staging developer-build --show-labels
NAME      STATUS    AGE    LABELS
dev        Active    19d    instrumentation.opentelemetry.io/inject-java=true,istio-injection=enabled,kubernetes.io/metadata.name=dev
staging    Active    19d    istio-injection=enabled,kubernetes.io/metadata.name=staging
developer-build Active    19d    istio-injection=enabled,kubernetes.io/metadata.name=developer-build
bingsul103@instance-20260620-064227:~$
```

Hình 4: Các namespace triển khai có label istio-injection=enabled

Kết quả kiểm tra pod trong namespace dev cho thấy mỗi pod ứng dụng có container nghiệp vụ và container istio-proxy. Trạng thái 2/2 Running chứng minh workload và sidecar đều sẵn sàng.

```
bingsul103@instance-20260620-064227:~$ kubectl get pods -n dev -o jsonpath='{range .items[*]}{.metadata.name}{ " containers="}{.spec.containers[*].name}{ "\n"}{end}'
backoffice-bff-77d8ff8f84-lbf7f containers=backoffice-bff istio-proxy
backoffice-ui-658cf45b8f-vdd7w containers=backoffice-ui istio-proxy
cart-798588f7f5-smvlf containers=cart istio-proxy
customer-69f679bcf-g62kw containers=customer istio-proxy
elasticsearch-coordinating-0 containers=elasticsearch istio-proxy
elasticsearch-data-0 containers=elasticsearch istio-proxy
elasticsearch-ingest-0 containers=elasticsearch istio-proxy
elasticsearch-ingest-1 containers=elasticsearch istio-proxy
elasticsearch-master-0 containers=elasticsearch istio-proxy
inventory-7d87d5bd64-wx6m5 containers=inventory istio-proxy
kafka-connect-689c6d7bcd-gl4s7 containers=kafka-connect istio-proxy
kafka-controller-0 containers=kafka istio-proxy
kafka-controller-1 containers=kafka istio-proxy
kafka-controller-2 containers=kafka istio-proxy
keycloak-0 containers=keycloak istio-proxy
location-7cbbd5468d-8dwjb containers=location istio-proxy
media-6b558b76f6-n64dx containers=media istio-proxy
order-6b187bc659-rgbb5 containers=order istio-proxy
payment-5fbd8dccc-fwnd6 containers=payment istio-proxy
postgres-postgresql-0 containers=postgresql istio-proxy
product-cb7f6047-hvlnj containers=product istio-proxy
redis-master-0 containers=redis istio-proxy
redis-replicas-0 containers=redis istio-proxy
redis-replicas-1 containers=redis istio-proxy
redis-replicas-2 containers=redis istio-proxy
sampledata-6df978bd7d-wrphj containers=sampledata istio-proxy
search-7d6b76c474-n6jpl containers=search istio-proxy
storefront-bff-76c54cb67-kbpzz containers=storefront-bff istio-proxy
storefront-ui-6f559678b6-265bt containers=storefront-ui istio-proxy
swagger-ui-5ff6c4cbd-c9cxm containers=swagger-ui istio-proxy
tax-7b8d7f7877-7t8rd containers=tax istio-proxy
bingsul103@instance-20260620-064227:~$
```

Hình 5: Pod trong dev có container ứng dụng và sidecar istio-proxy

```
bingsu1103@instance-20260620-064227:~$ kubectl get pods -n dev
NAME                                READY   STATUS    RESTARTS   AGE
backoffice-bff-77d8ff8f84-lbf7f    2/2     Running   0           21h
backoffice-ui-658cf45b8f-vdd7w     2/2     Running   0           21h
cart-798588f7f5-smvlf              2/2     Running   1 (21h ago)  21h
customer-69f679bcf-g62xw           2/2     Running   0           16m
elasticsearch-coordinating-0        2/2     Running   0           39h
elasticsearch-data-0                2/2     Running   1 (39h ago)  39h
elasticsearch-ingest-0              2/2     Running   0           39h
elasticsearch-ingest-1              2/2     Running   0           39h
elasticsearch-master-0              2/2     Running   0           39h
inventory-7d87d5bd64-wx6m5          2/2     Running   1 (21h ago)  21h
kafka-connect-689c6d7bcd-gl4s7      2/2     Running   0           17h
kafka-controller-0                  2/2     Running   10 (3h23m ago)  47h
kafka-controller-1                  2/2     Running   5 (3h24m ago)  47h
kafka-controller-2                  2/2     Running   8 (5h30m ago)  47h
keycloak-0                          2/2     Running   50 (2d20h ago)  6d23h
location-7cbbd5468d-8dwjb           2/2     Running   0           169m
media-6b558b76f6-n64dx              2/2     Running   0           21h
order-6bf87bc659-rgbb5              2/2     Running   0           21h
payment-5fbd8dccfc-twnd6            2/2     Running   1 (21h ago)   21h
postgres-postgresql-0               2/2     Running   0           21h
product-cb7ff6947-hvlnj             2/2     Running   0           3h31m
redis-master-0                      2/2     Running   0           21h
redis-replicas-0                    2/2     Running   124 (21h ago)  6d22h
redis-replicas-1                    2/2     Running   120 (21h ago)  6d22h
redis-replicas-2                    2/2     Running   122 (21h ago)  6d22h
sampledata-6df978bd7d-wrpjh         2/2     Running   0           21h
search-7d6b76c474-n6jpl            2/2     Running   0           91m
storefront-bff-76c54cbd67-kbpzz     2/2     Running   0           158m
storefront-ui-6f559678b6-265bt      2/2     Running   0           16m
swagger-ui-5ff6c4ccbd-c9cxm         2/2     Running   0           167m
tax-7b8d7f7877-7t8rd               2/2     Running   0           21h
bingsu1103@instance-20260620-064227:~$
```

Hình 6: Các pod ứng dụng trong dev ở trạng thái 2/2 Running

8.4 mTLS và DestinationRule

Namespace `dev` sử dụng `PeerAuthentication` ở chế độ `STRICT`. Cấu hình này yêu cầu traffic service-to-service trong mesh phải sử dụng mTLS. Các `DestinationRule` tương ứng cấu hình `ISTIO_MUTUAL` cho các service có sidecar, giúp client trong mesh sử dụng TLS do Istio quản lý khi gọi nội bộ.

```
bingus1103@instance-20260620-064227:~$ kubectl get peerauthentication -n dev
kubectl get destinationrule -n dev
kubectl describe peerauthentication dev-strict-mtls -n dev
NAME          MODE    AGE
dev-strict-mtls  STRICT  7d21h
NAME          HOST          AGE
backoffice-bff-istio-mutual  backoffice-bff.dev.svc.cluster.local  7d20h
backoffice-ui-istio-mutual   backoffice-ui.dev.svc.cluster.local    4d22h
cart-istio-mutual            cart.dev.svc.cluster.local             7d20h
customer-istio-mutual        customer.dev.svc.cluster.local          7d20h
inventory-istio-mutual       inventory.dev.svc.cluster.local         7d20h
media-istio-mutual           media.dev.svc.cluster.local            7d20h
order-istio-mutual           order.dev.svc.cluster.local            7d20h
payment-istio-mutual         payment.dev.svc.cluster.local           7d20h
product-istio-mutual         product.dev.svc.cluster.local           7d20h
sampledata-istio-mutual      sampledata.dev.svc.cluster.local        7d20h
search-istio-mutual          search.dev.svc.cluster.local            7d20h
storefront-bff-istio-mutual   storefront-bff.dev.svc.cluster.local    7d20h
storefront-ui-istio-mutual    storefront-ui.dev.svc.cluster.local     4d22h
swagger-ui-istio-mutual      swagger-ui.dev.svc.cluster.local        4d22h
tax-istio-mutual             tax.dev.svc.cluster.local              7d20h
Name:          dev-strict-mtls
Namespace:     dev
Labels:        environment=dev
Annotations:   argocd.argoproj.io/tracking-id: yas-dev:security.istio.io/PeerAuthentication:dev/dev-strict-mtls
API Version:   security.istio.io/v1beta1
Kind:          PeerAuthentication
Metadata:
  Creation Timestamp:  2026-07-01T09:31:41Z
  Generation:         1
  Resource Version:    1619595
  UID:                 c745e6e8-27a4-4ba5-b5e8-a9a04da2459d
Spec:
  Mtls:
    Mode:  STRICT
Events:   <none>
bingus1103@instance-20260620-064227:~$
```

Hình 7: PeerAuthentication STRICT và DestinationRule ISTIO_MUTUAL trong namespace dev

Cách cấu hình này giới hạn phạm vi áp dụng vào các service trong mesh, tránh ảnh hưởng tới thành phần hạ tầng hoặc workload không có sidecar.

8.5 Retry policy

Retry policy được triển khai bằng `VirtualService`. Các service quan trọng trong luồng nghiệp vụ như `product`, `cart`, `order`, `tax`, `payment` và `inventory` có cấu hình retry cho lỗi tạm thời. Ví dụ minh chứng là `tax-retry` với số lần thử lại và thời gian timeout trên từng lần thử.

```
bingsul103@instance-20260620-064227:~$ kubectl get virtualservice -n dev
kubectl describe virtualservice tax-retry -n dev
NAME          GATEWAYS          HOSTS          AGE
cart-retry    ["cart.dev.svc.cluster.local"]  ["cart.dev.svc.cluster.local"]  7d21h
inventory-retry ["inventory.dev.svc.cluster.local"] ["inventory.dev.svc.cluster.local"] 7d21h
order-retry    ["order.dev.svc.cluster.local"]  ["order.dev.svc.cluster.local"]  7d21h
payment-retry  ["payment.dev.svc.cluster.local"] ["payment.dev.svc.cluster.local"] 7d21h
product-retry  ["product.dev.svc.cluster.local"] ["product.dev.svc.cluster.local"] 7d21h
tax-retry      ["tax.dev.svc.cluster.local"]    ["tax.dev.svc.cluster.local"]    4d22h
yas-ingress-vs ["yas-gateway"]    ["storefront.dev.yas.local.com","identity.dev.yas.local.com","backoffice.dev.yas.local.com","swagger.dev.yas.local.com"] 14d
Name:          tax-retry
Namespace:     dev
Labels:        environment=dev
Annotations:    argocd.argoproj.io/tracking-id: yas-dev:networking.istio.io/VirtualService:dev/tax-retry
API Version:   networking.istio.io/v1beta1
Kind:          VirtualService
Metadata:
  Creation Timestamp: 2026-07-04T09:10:28Z
  Generation:        1
  Resource Version:   1619615
  UID:                9cf9ed7e-6bd2-434d-94d9-57557d969324
Spec:
  Hosts:
    tax.dev.svc.cluster.local
  Http:
    Retries:
      Attempts: 3
      Per Try Timeout: 2s
      Retry On: 5xx,reset,connect-failure,retriable-4xx
    Route:
      Destination:
        Host: tax.dev.svc.cluster.local
        Timeout: 10s
    Events: <none>
bingsul103@instance-20260620-064227:~$
```

Hình 8: VirtualService **tax-retry** cấu hình retry cho lỗi tạm thời

Retry policy giúp tăng độ ổn định của service-to-service traffic trong trường hợp lỗi mạng ngắn hạn hoặc backend phản hồi không ổn định trong thời gian ngắn.

8.6 AuthorizationPolicy và kiểm thử allow/deny

AuthorizationPolicy được dùng để giới hạn quyền truy cập giữa các service dựa trên ServiceAccount. Cách tiếp cận này phù hợp với microservices vì mỗi workload có ServiceAccount riêng, từ đó chính sách có thể định nghĩa rõ service nào được gọi service nào.

Nhóm kiểm thử hai trường hợp đại diện. Trường hợp hợp lệ sử dụng ServiceAccount **storefront-bff** gọi endpoint health của **product** và nhận HTTP 200. Trường hợp không hợp lệ sử dụng ServiceAccount **search** gọi **payment** và bị Istio RBAC chặn với HTTP 403.

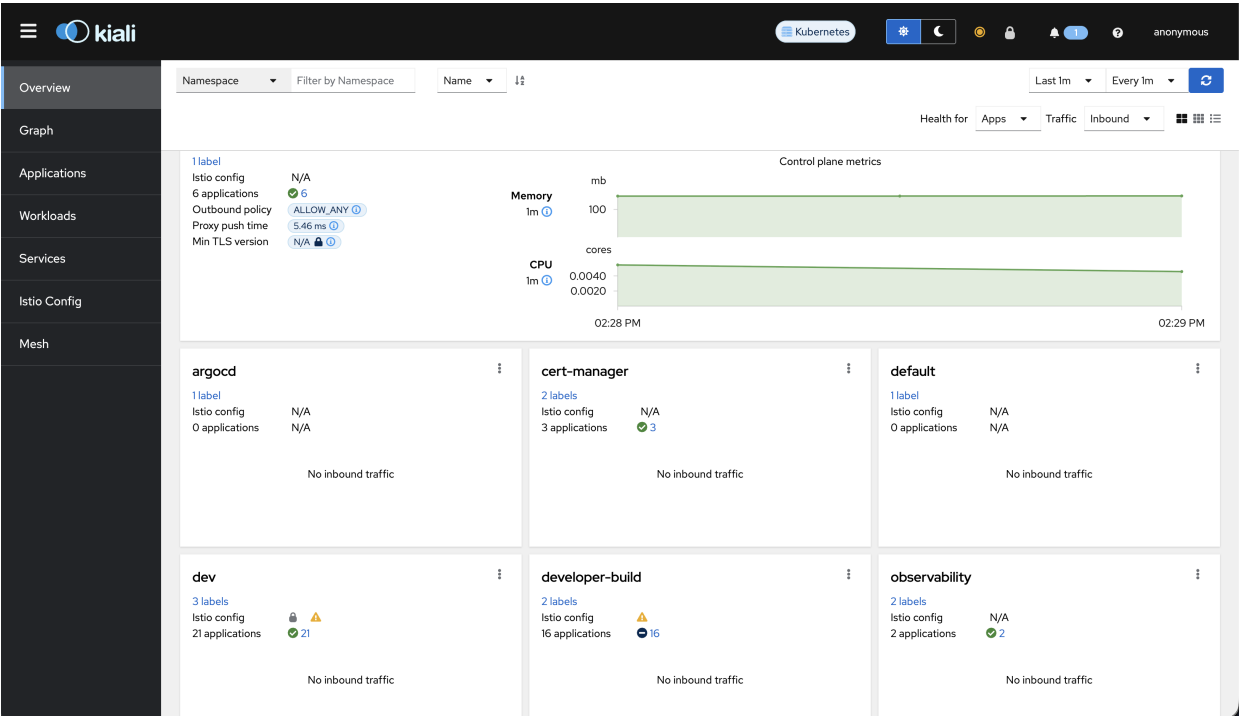
```
bingsul103@instance-20260620-064227:~$ kubectl delete pod curl-allowed curl-denied -n dev --ignore-not-found
kubectl run curl-allowed -n dev --image=curlimages/curl:8.8.0 --restart=Never \
--overrides='{ "spec": { "serviceAccountName": "storefront-bff", "containers": [ { "name": "curl", "image": "curlimages/curl:8.8.0", "command": ["sleep", "120"] } ] } }'
kubectl run curl-denied -n dev --image=curlimages/curl:8.8.0 --restart=Never \
--overrides='{ "spec": { "serviceAccountName": "search", "containers": [ { "name": "curl", "image": "curlimages/curl:8.8.0", "command": ["sleep", "120"] } ] } }'
kubectl wait --for=condition=Ready pod/curl-allowed pod/curl-denied -n dev --timeout=120s
kubectl exec -n dev curl-allowed -c curl -- \
curl -s -o /tmp/out -w '%{http_code}\n' \
http://product.dev.svc.cluster.local:8080/product/actuator/health
kubectl exec -n dev curl-denied -c curl -- \
curl -s -o /tmp/out -w '%{http_code}\n' \
http://payment.dev.svc.cluster.local:8081/payment/actuator/health
kubectl delete pod curl-allowed curl-denied -n dev --wait=false
pod/curl-allowed created
pod/curl-denied created
pod/curl-allowed condition met
pod/curl-denied condition met
200
403
pod "curl-allowed" deleted from dev namespace
pod "curl-denied" deleted from dev namespace
bingsul103@instance-20260620-064227:~$
```

Hình 9: Kiểm thử AuthorizationPolicy: request hợp lệ trả 200, request không hợp lệ bị chặn với 403 RBAC

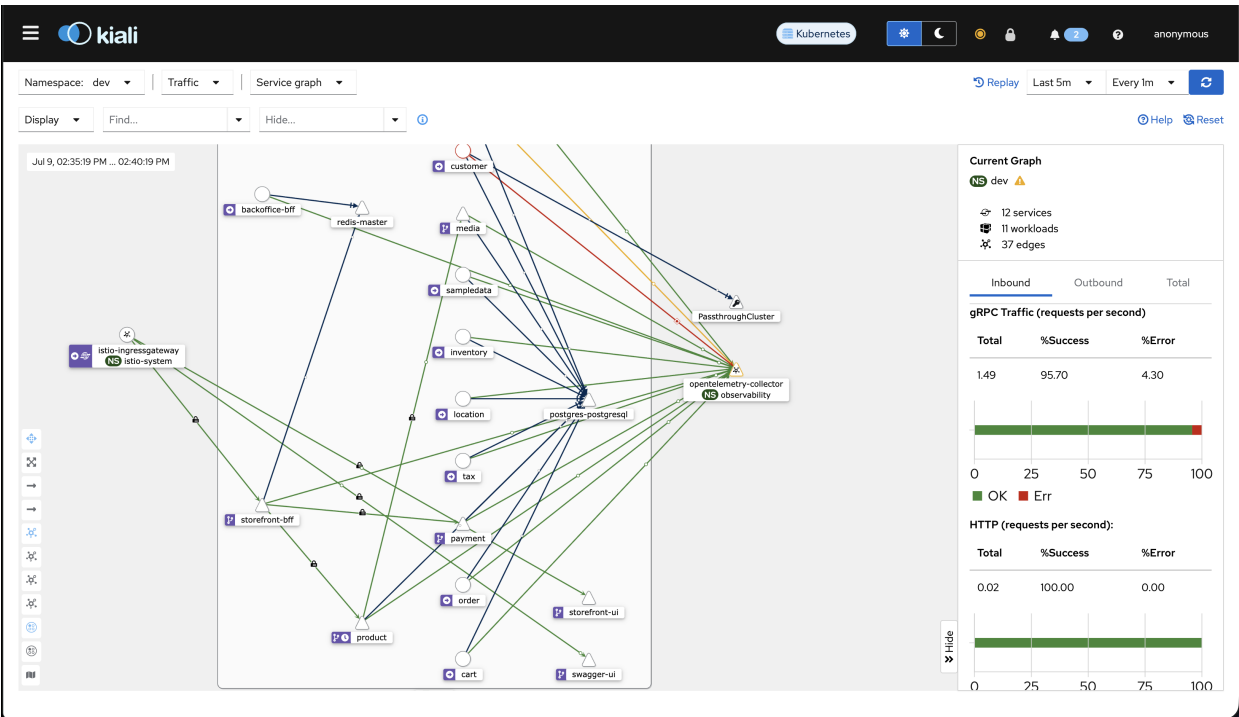
Kết quả này chứng minh policy không chỉ tồn tại ở manifest mà đã có hiệu lực ở runtime.

8.7 Kiali visualization

Kiali được sử dụng để quan sát topology, traffic và trạng thái bảo mật trong service mesh. Overview cho biết namespace nằm trong mesh và graph thể hiện quan hệ gọi giữa các service. Trong Service Graph, biểu tượng khóa trên các cạnh kết nối là minh chứng trực quan cho traffic được bảo vệ bằng mTLS.



Hình 10: Kiali Overview hiển thị các namespace trong mesh



Hình 11: Kiali Service Graph thể hiện topology, traffic và mTLS trong namespace dev

Kiali bổ sung góc nhìn vận hành cho phần cấu hình YAML. Thông qua graph, nhóm có thể kiểm tra service nào đang phát sinh traffic, kết nối nào có mTLS và workload nào cần điều chỉnh policy.

8.8 Kiểm chứng Istio ingress

Istio ingress gateway được kiểm chứng bằng cách port-forward service `istio-ingressgateway` và gửi request với Host header tương ứng. Các endpoint đại diện gồm Storefront, Swagger UI, Product categories và Payment providers đều trả HTTP 200.

```
bingsul103@instance-20260620-064227:~$ curl -s -o /tmp/storefront.html -w '%{http_code}\n' -H 'Host: storefront.dev.yas.local.com' http://127.0.0.1:18080/
curl -s -o /tmp/swagger.html -w '%{http_code}\n' -H 'Host: swagger.dev.yas.local.com' http://127.0.0.1:18080/swagger-ui/
curl -s -o /tmp/categories.json -w '%{http_code}\n' -H 'Host: storefront.dev.yas.local.com' http://127.0.0.1:18080/api/product/storefront/categories
curl -s -o /tmp/payment-providers.json -w '%{http_code}\n' -H 'Host: storefront.dev.yas.local.com' http://127.0.0.1:18080/api/payment/storefront/payment-providers
storefront: 200
swagger: 200
product categories: 200
payment providers: 200
bingsul103@instance-20260620-064227:~$
```

Hình 12: Kiểm chứng ingress: các endpoint chính đều trả HTTP 200

Kết quả này xác nhận Gateway và VirtualService đã định tuyến đúng request từ ingress vào các service trong hệ thống.

8.9 Kết luận phần Service Mesh

Phần Service Mesh đã hoàn thành các yêu cầu chính: sidecar injection, mTLS, DestinationRule, retry policy, AuthorizationPolicy, kiểm thử allow/deny, Kiali visualization và ingress routing. Các minh chứng bằng hình ảnh cho thấy cấu hình trong `dev` đã được apply và có hiệu lực ở runtime. Overlay `staging` đã được tách riêng trong repo GitOps để áp dụng cùng nhóm chính sách mesh cho môi trường tiền triển khai mà không phụ thuộc trực tiếp vào cấu hình namespace `dev`.

9 Observability

Nhóm đã triển khai bổ sung Observability trong namespace `observability`. Stack gồm Prometheus cho metrics, Grafana cho dashboard, Loki và Promtail cho log, Tempo cho distributed tracing và OpenTelemetry Collector cho việc tiếp nhận, xử lý, chuyển tiếp telemetry.

Phần này là khung để bổ sung hình ảnh sau khi nhóm chụp UI Grafana. Các minh chứng cần ưu tiên gồm: Helm releases trong namespace `observability`, trạng thái pod/service, Prometheus targets hoặc query, Grafana datasources, JVM dashboard, Loki logs và Tempo traces. Nội dung chi tiết được chuẩn bị trong file `docs/observability-report.md`.

10 Kết Luận

Đồ án đã xây dựng được quy trình triển khai theo hướng CI/CD và GitOps cho hệ thống YAS. Jenkins đảm nhiệm build/test/push image và cập nhật manifest. ArgoCD đồng bộ trạng thái triển khai từ repo GitOps vào cluster K3s. Istio được sử dụng để bổ sung Service Mesh cho môi trường `dev` và `staging`, bao gồm mTLS, retry, AuthorizationPolicy và ingress routing. Kiali và Observability stack cung cấp minh chứng trực quan cho trạng thái vận hành của hệ thống.

Kết quả hiện tại cho thấy nhóm đã triển khai được pipeline và hạ tầng cần thiết để vận hành các workload chính trong scope. Các bước tiếp theo khi hoàn thiện báo cáo là bổ sung đầy đủ hình ảnh minh chứng cho phần hạ tầng, CI/CD, GitOps và Observability, sau đó rà soát lại số hiệu hình, caption và trạng thái cluster tại thời điểm nộp.